Lab 4

ECE 455 / ECE 555, Fall 2026

Assigned	Friday, October 16
Due	**Friday, October 30, 11:59 p.m.**, through Canvas

Objective

- To set up the ADC
- To understand the concept of calibration for the sensor
- To interface an external analog sensor

Teamwork

Laboratory work is done in teams of two, formed in the first week and kept for the whole semester. Only one lab report is required for each lab team. But both members must be able to explain every part of the submission to the TA. Every lab in this course is demonstrated in person to the TA during TA office hours. Bring your board, your circuit, and your results. The TA checks the operation, verifies that it meets the assigned functionality for each individual student. Both team members demonstrate, and either one may be asked to explain any part of the submission. TA office hours are posted on Canvas. Plan your demonstration in advance rather than in the last session before the due date.

Preparation

- You will need a Tiva LaunchPad and a PC with the Keil µVision IDE installed
- A breadboard for circuit connections and a slide potentiometer
- An NTC thermistor (Vishay NTCLE100E3103JB0), and one fixed resistor from the general assortment to form a voltage divider

Background Setup

- Check the file UART.c for some functions used earlier in Lab 3. In this lab the given UART.c runs at 115200 baud, so set PuTTY to 115200 rather than the 9600 of Lab 3.
- The project folder has two parts, PART1 and PART2. You should build a Keil project for each part separately.
- Submit the points labeled R1 and R2 in a single PDF.

Part 1. Temperature Interfacing (60 points)

You will use the folder PART1 from the project folder.

1A. Set up the functions (24 points)

The Tiva LaunchPad chip, the TM4C123GH6PM, has an inbuilt temperature sensor. You will configure the ADC to read this temperature sensor, along with some functions that convert the raw ADC value to a temperature.

Function	Description
In ADCCode.c	
ADC0_Temp_Init()	The code to read the temperature from the sensor through ADC0. Configure it to read at 500 K samples/sec and use sequencer 3
ADC0_In()	Busy-wait ADC read
In main.c	
Convert_Raw_To_V()	Convert a raw ADC value to mV
Convert_V_To_C()	Use the equation given in the comment to convert voltage in mV to temperature in Celsius
Get_TempC()	The function calls Convert_Raw_To_V() and Convert_V_To_C() to return the temperature in Celsius
Convert_C_To_F()	Convert temperature from Celsius to Fahrenheit. The conversion equation is given in the comment

1B. Submit the code that shows the following behavior (16 points)

- Initialize ADC0 to read the temperature sensor
- The code should wait for the user to enter a key

Key pressed from the PC keyboard	Description
'c' or 'C'	Print the temperature in Celsius on PuTTY
'f' or 'F'	Print the temperature in Fahrenheit on PuTTY
Other than the above	Display "Press Valid key!"

1C. Set up the circuit and the code (12 points)

In this part you will add an external temperature sensor and check it against the internal one.

- Build a voltage divider from the NTC thermistor and one fixed resistor (R_{fixed}) with the fixed resistor from VCC to PE3 and the thermistor from PE3 to GND, and connect its midpoint to PE3 (AIN0).
- Draw the circuit diagram and submit a photograph of your prototype. State the fixed resistor value you chose and why. **R1**
- Parameters of the thermistor (Vishay NTCLE100E3103JB0) are:

Symbol	Value	Meaning
R0	10 000 Ω	Resistance at the reference temperature T0
T0	298.15 K (25 °C)	Reference temperature
B	3977 K	The B (Beta) constant, which sets how fast the resistance falls with temperature

- Write the following functions:

Function	Description
In ADCCode.c	
ADC0_Thermistor_Init()	Configure ADC0 sequencer 2 to read PE3 (AIN0), software trigger, so that it can be used alongside the sequencer 3 temperature-sensor channel of Part 1
ADC0_Thermistor_In()	Busy-wait read of sequencer 2
In main.c	
Convert_Thermistor_To_C()	Convert the raw ADC sample to a voltage, then to the thermistor resistance from the divider equation, then to a temperature in Celsius using the Beta equation 1/T = 1/T0 + (1/B) ln(R/R0), with T and T0 in kelvin
Get_ThermistorC()	calls ADC0_Thermistor_In() and Convert_Thermistor_To_C() and returns the temperature in Celsius

Calculation for the thermistor, in three steps, inside Convert_Thermistor_To_C():

1. Voltage at the midpoint, from the 12-bit ADC sample:
$$V = \frac{sample}{4096} \times 3.3$$
2. Thermistor resistance, from the divider:
$$R = \frac{V \times R_{fixed}}{3.3 - V}$$
3. Temperature, from the Beta equation, with T and T0 in kelvin:
$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \times \ln \frac{R}{R_0}$$

Use float and logf() from <math.h> for step 3.

1D. Demonstration (8 points)

Extend the key-press menu from Part 1B so that, on the same 'c' or 'f' key press, the program prints both readings on the next line: the internal sensor's temperature (Part 1B) and the external thermistor's temperature (Part 1C), in the requested unit.

- Submit a short discussion in the report of how closely the two agree, and name one source of the difference (self-heating, thermal contact with the board, ADC quantization, or the assumed B value). **R2**

Part 2. ADC Interfacing with an External Slide Potentiometer (40 points)

2A. Set up the circuit

You should set up the following. There will be three thin pins and four thick pins (located on a longer edge and symmetric in position). You can bend the thick pins by 90 degrees towards the outside.

- Connect the pin labeled 1 to GND, 2 to PE2, and 3 to VCC on the LaunchPad.



You will use the folder PART2 from the project folder.

2B. Complete the following functions (20 points)

Function	Description
Inside ADCCode.c	
ADC0_InitSWTriggerSeq3_Ch1()	Configure ADC0 to read the voltage at PE2 at 125 K samples/sec using sequencer 3
Inside Main.c	
unsigned long Convert()	Convert a 12-bit binary ADC sample into a 32-bit unsigned fixed-point distance. The formula for conversion and calibration is given as a comment and you need to code it
SysTick_Init_Interrupts()	Configure SysTick to generate an interrupt every 500 ms
SysTick_Handler()	The code reads the ADC reading and sets the global variable "Flag" equal to 1

2C. Submit the code that shows the following behavior (20 points)

- The code initializes the PLL, the UART, the ADC, and SysTick_Init_Interrupts()
- The code should print the distance value every 500 ms

Note:

- Your code can read the status of the global variable "Flag". When it is set, it first disables it and then prints the value.
- The function ConvertDistancetostring() is pre-written and will convert the fixed-point distance to a string to be printed via the UART. You need not modify it.
- You can use the function UART_printf() to print the string value of the distance.

Report

- R1 and R2 will come only in the report. You are free to add information that can help the TA in grading, however, we do not expect any extra credit for writing background theory information. This should be submitted as Report4.pdf inside the .zip file.
- The one submission report should display the names of both students and contributions of each student.
- A single Keil µVision project folder with all code.
- Code should be commented for better understanding and to highlight any design choices or calculations that you made.
- The code should be properly structured and should make sense. Poorly structured code, though working, may severely affect your score.
- In Embedded Systems, it is preferred that your code should be as simple as possible so that the translation into binary code is shorter in size and portable. Writing unnecessary compiler-specific directives is not a good idea.
- The report and the Keil µVision project folder should be submitted in a single .zip folder.

AI Use Statement

Generative AI tools are permitted on this lab, but their use must be disclosed. End your report with a short AI Use Statement naming the tool, what you used it for, and what it contributed. If you used no AI tool, write "No generative AI tools were used."

Late Work

Submissions are due at 11:59 p.m. on the stated date. **Late submissions are not accepted.** If you must miss a submission date, inform the instructor in advance, or submit a legitimate, documented reason for the absence. Failure to do so results in a grade of zero for this assignment. See the syllabus for the full policy.